

Flask & MongoDB — Assignment Submission

Aditya Bawane

Project Structure

```
Flask-MongoDB/  
├── app.py  
├── data.json  
├── requirements.txt  
├── .env  
├── .gitignore  
└── templates/  
    ├── index.html  
    └── success.html
```

1. /api Route — Serve JSON from a Backend File

Task: A Flask route that reads data.json and returns it as a JSON response.

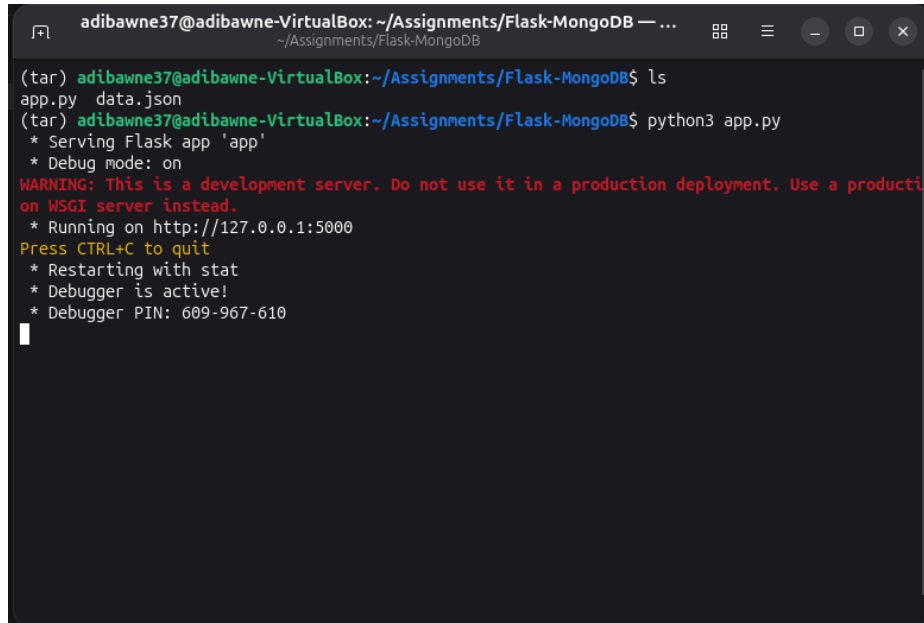
data.json

```
[  
  {  
    "id": 1,  
    "name": "Aditya",  
    "course": "Information Technology"  
  },  
  {  
    "id": 2,  
    "name": "Sumit",  
    "course": "Computer Engineering"  
  }  
]
```

app.py — /api route

```
@app.route('/api')  
def api():  
    with open('data.json', 'r') as file:  
        data = json.load(file)  
    return jsonify(data)
```

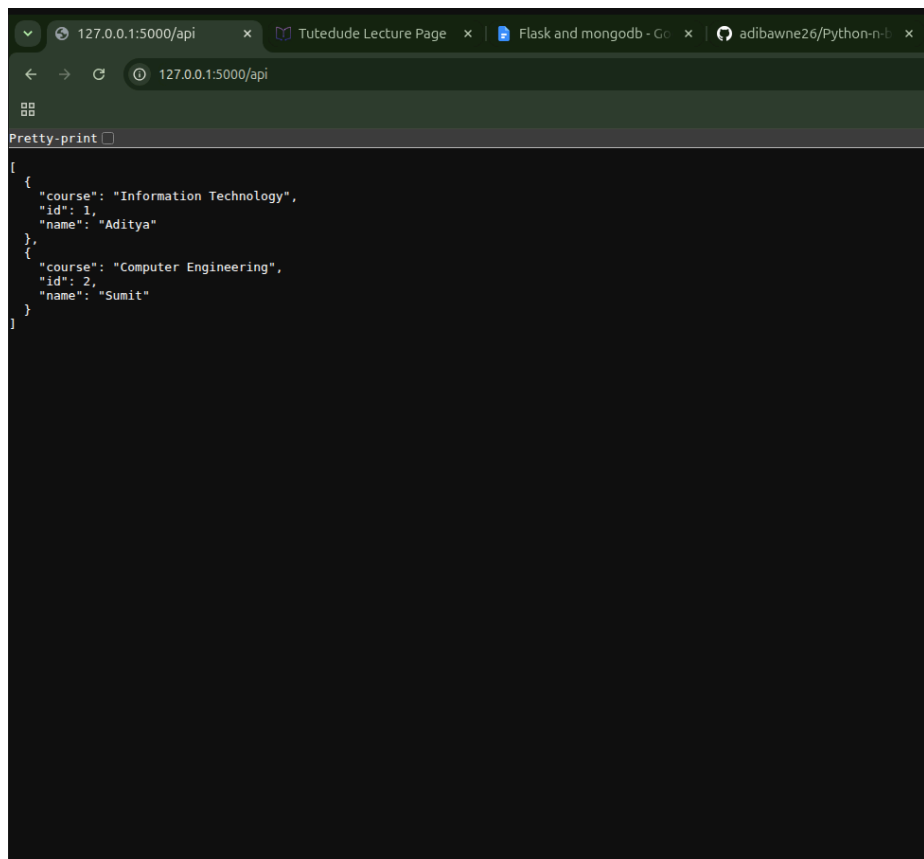
Explanation: The route opens data.json, parses it with json.load(), and returns it via jsonify() so the browser receives a proper JSON response.



A terminal window titled 'adibawne37@adibawne-VirtualBox: ~/Assignments/Flask-MongoDB' shows the following commands and output:

```
(tar) adibawne37@adibawne-VirtualBox:~/Assignments/Flask-MongoDB$ ls
app.py  data.json
(tar) adibawne37@adibawne-VirtualBox:~/Assignments/Flask-MongoDB$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 609-967-610
```

Fig 1.1 — Flask dev server running (*python3 app.py*)



A web browser window with the address bar showing '127.0.0.1:5000/api' displays a JSON response. The response is formatted with 'Pretty-print' and shows a list of two objects:

```
[
  {
    "course": "Information Technology",
    "id": 1,
    "name": "Aditya"
  },
  {
    "course": "Computer Engineering",
    "id": 2,
    "name": "Sumit"
  }
]
```

Fig 1.2 — GET /api returning the JSON list from data.json

2. Form Submission to MongoDB Atlas

Task: A frontend form that inserts data into MongoDB Atlas on submit, redirecting to a success page, or showing errors inline without redirecting.

app.py — MongoDB connection & routes

```
from flask import Flask, render_template, request, redirect, jsonify
from pymongo import MongoClient
from dotenv import load_dotenv
import json, os

load_dotenv()
app = Flask(__name__)

client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DATABASE_NAME")]
collection = db[os.getenv("COLLECTION_NAME")]

@app.route('/')
def home():
    return render_template("index.html")

@app.route('/submit', methods=['POST'])
def submit():
    try:
        student = {
            "name": request.form['name'],
            "email": request.form['email']
        }
    except Exception as e:
        error_message = f"An error occurred: {str(e)}"
        return render_template("index.html", error=error_message)

    collection.insert_one(student)
    return redirect('/success')

@app.route('/success')
def success():
    return render_template("success.html")
```

Explanation: MongoClient connects to Atlas using the URI from .env. /submit reads the form fields into a dict, inserts it via collection.insert_one(), then redirects to /success. Any exception is caught and re-rendered on index.html as an error message instead of redirecting.

templates/index.html (form)

```
<h2>Student Registration</h2>

{% if error %}
<p style="color:rgb(233, 233, 123);">{{ error }}</p>
{% endif %}

<form action="/submit" method="POST">
```

```
<label>Name</label><br>
<input type="text" name="name" required><br><br>
<label>Email</label><br>
<input type="email" name="email" required><br><br>
<input type="submit" value="Submit">
</form>
```

Explanation: A simple POST form for name and email. The `{% if error %}` block conditionally displays a validation/DB error passed back from Flask, on the same page.

templates/success.html

```
<h2>Data submitted successfully!</h2>
```

Explanation: Rendered by the `/success` route after a successful insert.

.env (credentials — sanitised)

```
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.cg3nxhr.mongodb.net/?appName=Cluster
0
DATABASE_NAME=college
COLLECTION_NAME=students
```

Explanation: Stores the Atlas connection string and target database/collection names, loaded via `load_dotenv()` so credentials never appear in source code.

requirements.txt

```
flask
pymongo
dnspython
python-dotenv
```

.gitignore

```
.env
```

Explanation: Excludes `.env` from version control so Atlas credentials are never pushed to GitHub.

Student Registration

Name
Prashant

Email
prash@gmail.com

Submit

Fig 2.1 — Student Registration form (index.html)

Data submitted successfully!

Fig 2.2 — Redirected to /success after a valid submission

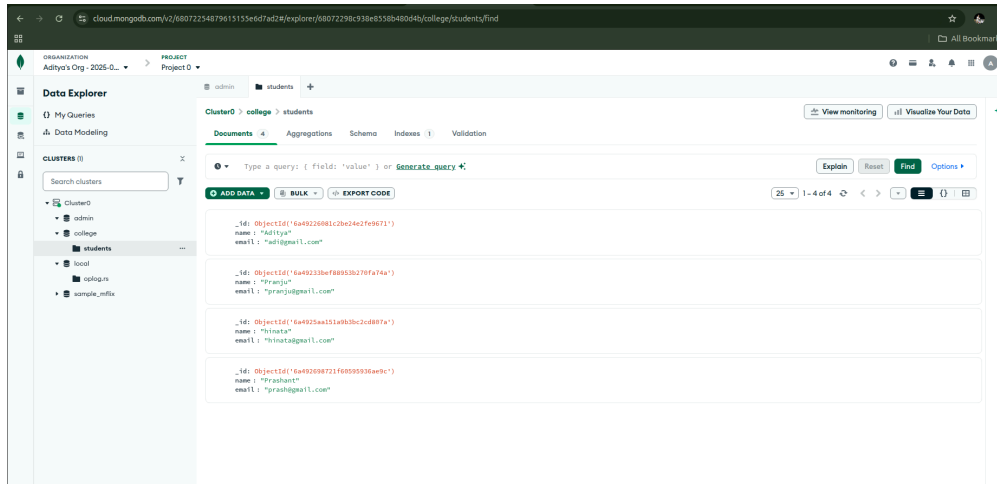


Fig 2.3 — MongoDB Atlas Data Explorer showing inserted documents in college.students